

BLOC A • SÉANCES 1 À 3

Démystifier

Du vocabulaire au choix d'un modèle



Les séances de ce bloc

1

Séance 1

IA, ML, DL : remettre les mots à leur place

2

Séance 2

Sous le capot d'un LLM, sans mathématiques

3

Séance 3

Panorama des modèles et critères de choix

SÉANCE 1 • DÉMYSTIFIER

IA, ML, DL : remettre les mots à leur place

Ce qu'est vraiment un modèle, et ce qu'il ne fait pas

Au programme aujourd'hui

1

Définir

Qu'est-ce qu'un modèle d'intelligence artificielle ?

2

Situer

IA, machine learning, deep learning, IA générative

3

Comprendre

Comment un modèle est fabriqué, avant qu'on l'utilise

4

Utiliser

Les tâches réellement exploitables depuis une app web

5

Douter

Ce que l'IA ne sait pas faire, et pourquoi

Un modèle, c'est quoi au juste ?

- Une fonction : elle prend une entrée, elle rend une sortie.
- Ses paramètres n'ont pas été écrits à la main : ils ont été appris.
- Apprendre veut dire ajuster des milliards de nombres sur des exemples.
- Une fois appris, le modèle est figé : un fichier, et du calcul.



Un modèle est un fichier, pas un cerveau

Le modèle que vous ferez tourner en séance 4 est un fichier de 2 Go sur votre disque. Rien de plus.

Qu'est-ce que le Machine Learning ?

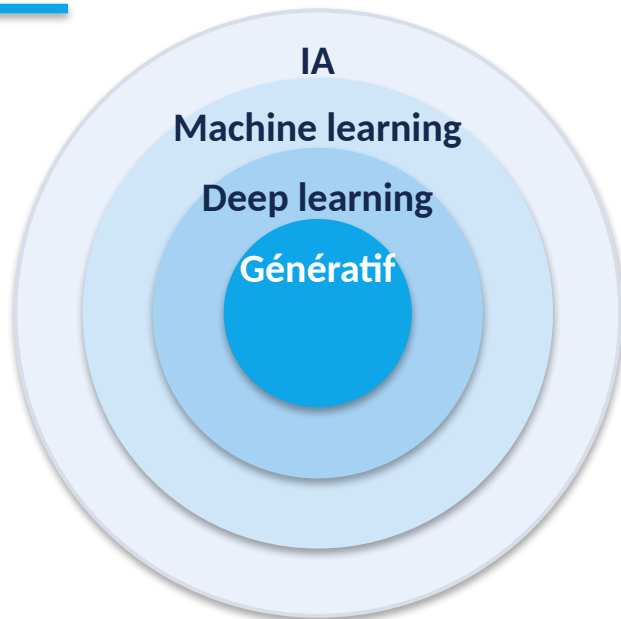
- Apprentissage automatique des algorithmes à partir de données.
- Le but : prédire, classifier ou optimiser des actions.
- On ne programme pas la règle : on l'apprend des exemples.



ML = Data + Algo

Des données + un algorithme qui apprend = un modèle capable de généraliser.

Des poupées russes, pas des synonymes



-  **IA** : toute technique qui imite une tâche humaine
-  **Machine learning** : apprendre des règles à partir de données
-  **Deep learning** : apprendre avec des réseaux de neurones
-  **Génératif** : produire du texte, une image, du son

La maison de l'IA, du data à l'agentique



Tout repose sur les données : sans elles, aucun étage ne tient debout.

Ce cours travaille aux étages du haut : IA générative, RAG, agents.

Tous reposent sur le ML du rez-de-chaussée : d'où ce détour par ses bases.

Ce que vous faites déjà, et ce qui change

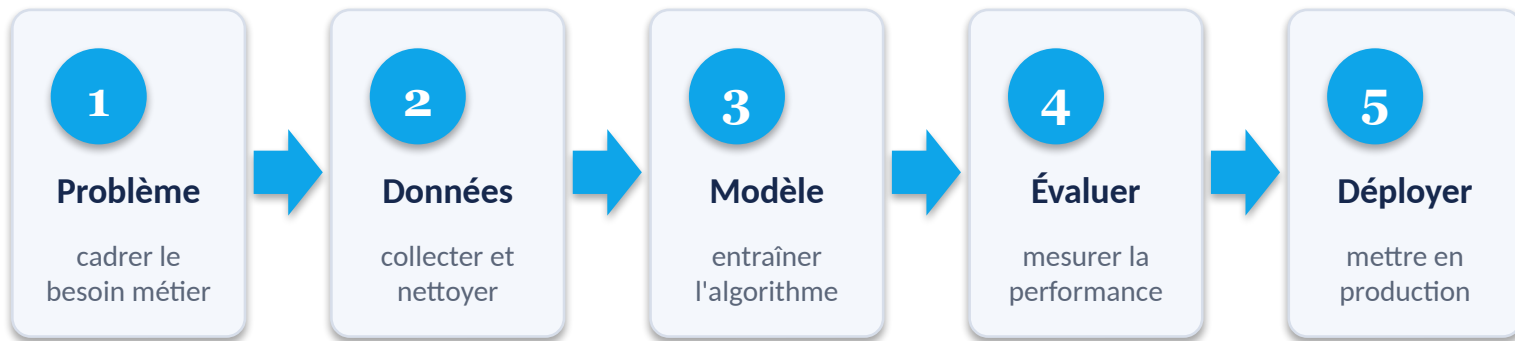
Programmation classique

- Vous écrivez la règle
- Entrée + règles donnent une sortie
- Résultat identique à chaque appel
- Un bug se trouve et se corrige
- Testable au sens habituel

Modèle appris

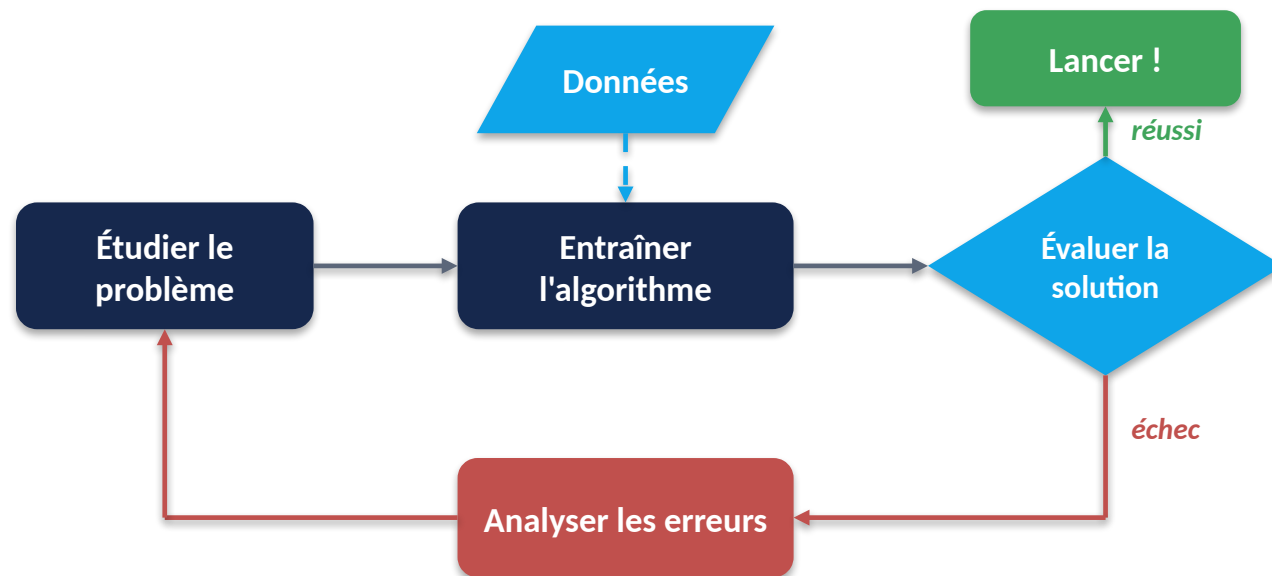
- La règle est apprise, jamais écrite
- Entrée + poids donnent une sortie probable
- Deux appels identiques peuvent différer
- Une erreur se contourne, rarement se corrige
- Testable seulement statistiquement

Le pipeline d'un projet ML



Dans ce cours, tout ce pipeline a déjà eu lieu : vous récupérez le modèle fini, à l'étape 5.

Un cycle itératif



- **Mesurer avant de lancer**
un modèle non évalué ne part pas en production
- **Les données d'abord**
sans elles, rien ne s'entraîne
- **L'erreur informe**
chaque échec dit quoi corriger

Le ML n'est pas linéaire : on revient en arrière dès qu'une étape échoue.

Trois façons de brancher un modèle sur une app



Générer

Produire du texte : résumé, reformulation, réponse, brouillon de mail, description produit.



Classer

Ranger une entrée dans une case : sentiment, catégorie de ticket, modération, détection de spam.



Vectoriser

Transformer un texte en vecteur pour comparer du sens : recherche interne, suggestions, RAG.

Le besoin dicte le modèle, pas l'inverse

Besoin produit	Ce qu'il faut	Taille typique
Résumer un ticket	Modèle de génération	3B suffit
Modérer un commentaire	Classifieur ou petit LLM	1B à 3B
Recherche interne	Modèle d'embedding	moins de 1 Go
Chatbot support	Génération + RAG	7B et plus
Texte alternatif d'image	Modèle vision	spécialisé

Un gros modèle pour une tâche simple est un gâchis de latence et d'argent.

Ce qu'un modèle de langage ne fait pas

Quatre confusions à évacuer tout de suite.

- **Il ne comprend pas** : il calcule le mot le plus probable après les précédents, sans notion de sens
- **Il ne sait pas ce qui est vrai** : rien dans son calcul ne distingue un fait d'une invention plausible
- **Il n'a pas de mémoire** : chaque appel repart de zéro, sauf si vous renvoyez l'historique vous-même
- **Il n'accède à rien** : ni à votre base, ni au web, tant que vous ne lui donnez pas explicitement les moyens

Un modèle, vu depuis votre métier

Pas de magie : une requête HTTP, un JSON en entrée, un JSON en sortie.

```
# le modele tourne sur votre machine, il repond en HTTP
curl http://localhost:11434/api/generate -d '{
  "model": "qwen2.5:3b",
  "prompt": "Resume ce ticket en une phrase : ...",
  "stream": false
}'

{"response": "Le client ne parvient pas a se connecter...",
 "eval_count": 24, "eval_duration": 8400000000}
```

C'est exactement le contrat d'API que vous manipulez déjà. Nous l'installerons en séance 4.

Démystifier l'IA

- ✓ Un modèle est une fonction dont les paramètres ont été appris, pas écrits.
- ✓ IA englobe le ML, qui englobe le DL, qui porte l'IA générative.
- ✓ Un modèle naît d'un cycle itératif : données, entraînement, évaluation.
- ✓ Trois usages web : générer, classer, vectoriser.
- ✓ Le modèle ne comprend pas, ne sait pas, ne se souvient pas, n'accède à rien.

SÉANCE 2 • DÉMYSTIFIER

Sous le capot d'un LLM, sans mathématiques

Tokens, vecteurs, probabilités : d'où vient la réponse

Au programme aujourd'hui

1

Découper

La tokenisation, et pourquoi elle vous est facturée

2

Représenter

Les embeddings : du sens dans des nombres

3

Prédire

Le modèle joue à l'autocomplétion, très bien

4

Régler

Température, contexte, et l'origine des hallucinations

Ce qui se passe entre votre requête et la réponse

1

Tokeniser

Le texte est découpé en morceaux, les tokens.

2

Vectoriser

Chaque token devient un vecteur de nombres.

3

Prédire

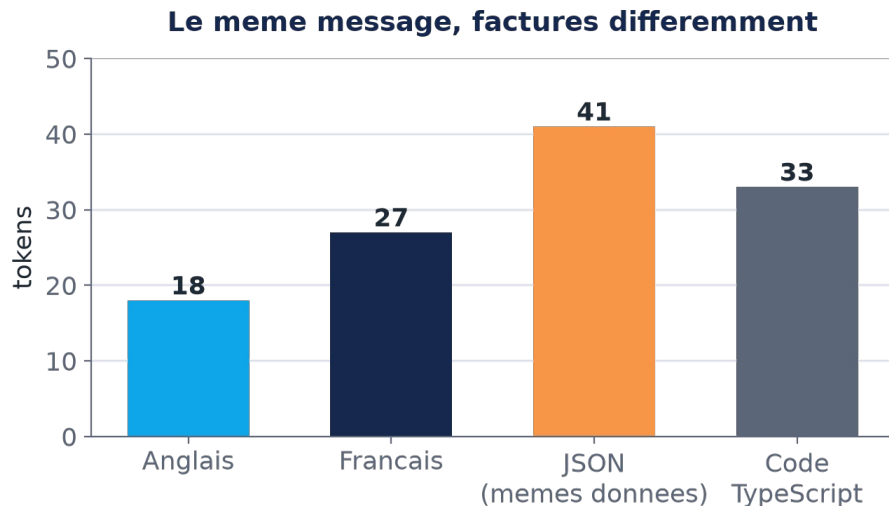
Le modèle calcule une probabilité pour chaque token suivant.

4

Tirer

Un token est choisi, ajouté, et on recommence.

Le français vous coûte plus cher que l'anglais



Un token vaut environ 4 caractères en anglais, souvent moins en français.

Les accents et les mots longs se découpent en plusieurs morceaux.

Pourquoi cette histoire de tokens vous concerne

- Un fournisseur cloud facture au token, entrée et sortie séparément.
- En local, le token reste l'unité de mesure de la vitesse et de la mémoire.
- Envoyer du JSON verbeux double la facture pour la même information.

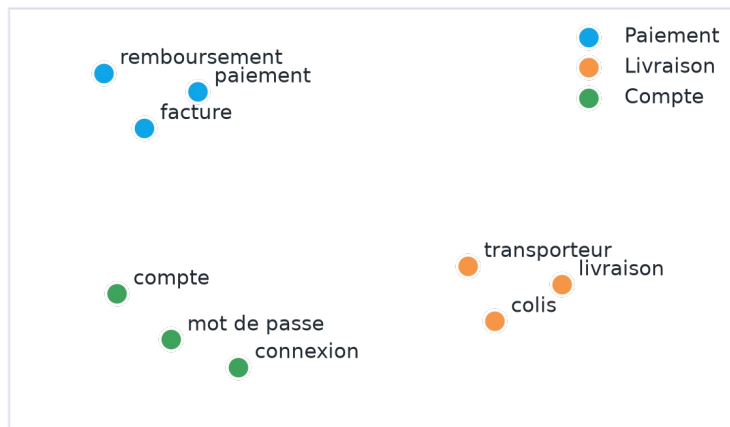


Le token est votre unité d'oeuvre

Compter les tokens avant d'envoyer est un réflexe d'ingénieur, au même titre que mesurer le poids d'un bundle.

Transformer du sens en coordonnées

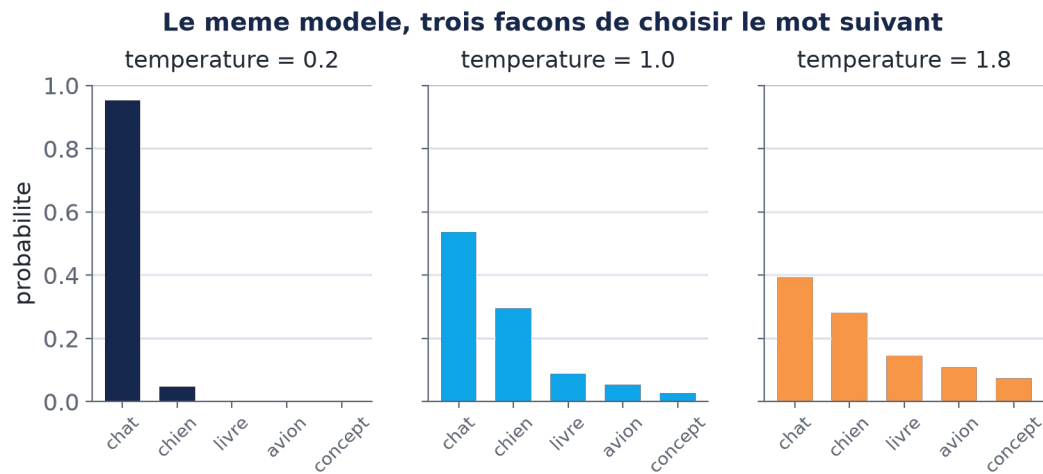
Des tickets de support projetés en 2 dimensions



Proches = memes sujets. C'est tout ce qu'un moteur de recherche semantique exploite.

Deux textes proches en coordonn  es parlent du m  me sujet, m  me sans aucun mot commun.

La température décide du risque pris



À 0.2 le modèle joue la sécurité, à 1.8 il part en exploration.

Extraction et classification : température basse. Création : plus haute.

La fenêtre de contexte, votre vraie contrainte

Tout ce que le modèle « sait » pendant un appel tient dans cette fenêtre.

- **Elle est finie** : quelques milliers à quelques centaines de milliers de tokens selon le modèle
- **Elle contient tout** : consigne système, historique, documents joints et réponse en cours
- **Elle se remplit vite** : un historique de chat non taillé la sature en quelques dizaines de messages
- **Elle déborde en silence** : le début est tronqué, et le modèle oublie vos consignes sans prévenir

Pourquoi il invente, et pourquoi c'est normal

- Le modèle produit toujours le texte le plus plausible, jamais le plus vrai.
- Une invention bien formulée a exactement le même score qu'un fait exact.
- Il n'existe aucun réglage qui supprime les hallucinations.



Ce n'est pas un bug, c'est le principe

On ne corrige pas une hallucination : on l'encadre. Fournir les sources (RAG), contraindre le format, et vérifier côté serveur.

Les réglages, vus depuis votre code

```
// extraction de donnees : on veut du deterministe
const res = await fetch(`${BASE}/v1/chat/completions`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    model: 'qwen2.5:3b',
    temperature: 0,           // pas de creativite ici
    max_tokens: 200,          // plafond de cout et de latence
    messages: [{ role: 'user', content: prompt }],
  }),
});
```

`temperature: 0` ne garantit pas un résultat identique à chaque appel, mais s'en approche beaucoup.

Sous le capot d'un LLM

- ✓ Le texte est découpé en tokens : c'est l'unité de facturation et de mesure.
- ✓ Les embeddings placent le sens dans un espace où la distance est comparable.
- ✓ Le modèle prédit le token suivant, puis recommence : rien de plus.
- ✓ Température, contexte fini et plausibilité expliquent les hallucinations.

SÉANCE 3 • DÉMYSTIFIER

Panorama des modèles et critères de choix

Savoir arbitrer avant d'écrire la moindre ligne

Au programme aujourd'hui

1

Cartographier

Les familles de modèles et ce qu'elles savent faire

2

Opposer

Modèles propriétaires et modèles à poids ouverts

3

Mesurer

Ce que la taille change vraiment

4

Décider

Les cinq critères qui tranchent en entreprise

Un modèle par famille de tâche

Famille	Entrée et sortie	Usage web typique
Génération	texte vers texte	résumé, rédaction, chat
Embedding	texte vers vecteur	recherche sémantique, RAG
Vision	image vers texte	alt text, extraction de facture
Audio	son vers texte	transcription de réunion
Reranking	liste vers liste triée	affiner une recherche

Propriétaire ou poids ouverts : l'arbitrage de fond

Modèle propriétaire (API)

- Qualité de pointe immédiate
- Aucune infrastructure à gérer
- Facturé à l'usage, sans plancher
- Vos données partent chez un tiers
- Le modèle peut changer sous vos pieds

Modèle à poids ouverts

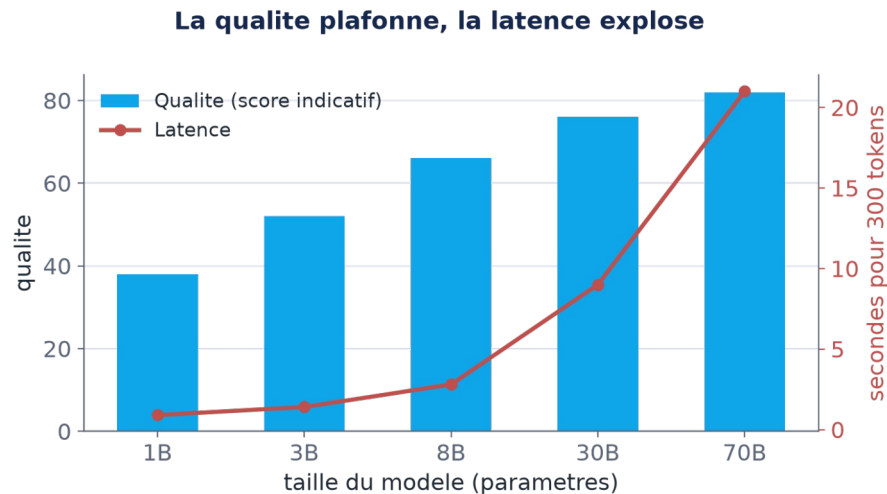
- Qualité inférieure à taille égale
- Vous gérez la machine et la charge
- Coût fixe, indépendant du volume
- Les données ne sortent jamais
- Version figée, reproductible

« Open source » veut dire trois choses distinctes

Vérifiez laquelle vous est réellement accordée avant de vous engager.

- **Les poids** : le fichier du modèle est téléchargeable : c'est le cas le plus courant
- **La licence** : elle peut interdire l'usage commercial ou imposer des conditions au-delà d'un seuil
- **Les données d'entraînement** : presque jamais publiées, même pour les modèles dits ouverts
- **Conséquence** : « poids ouverts » est le terme honnête ; « open source » est un raccourci marketing

Ce que vous payez quand vous prenez plus gros



Passer de 8B à 70B gagne quelques points de qualité et multiplie la latence par sept.

La bonne question n'est pas « quel est le meilleur », mais « lequel suffit ».

Les classements, et ce qu'ils ne vous disent pas

- Un benchmark mesure une tâche moyenne, jamais la vôtre.
- Les jeux de test finissent dans les données d'entraînement : c'est la contamination.
- Deux points d'écart au classement sont invisibles pour vos utilisateurs.



Le seul benchmark qui compte est le vôtre

Vingt cas réels tirés de votre produit valent mieux que n'importe quel classement public.
Nous le ferons en séance 11.

Les critères qui tranchent vraiment

1

Qualité suffisante

Non pas la
meilleure : celle
qui passe vos cas
de test.

2

Latence tenable

Ce que l'utilisateur
accepte d'attendre
devant son écran.

3

Coût au volume

Le prix à 10 000
utilisateurs, pas à
la démo.

4

Confidentialité

Ce que vous avez le
droit d'envoyer
dehors.

Choisir un modèle et le défendre

- 1 Chaque groupe reçoit un des cinq besoins produit du tableau.
- 2 Choisir une famille, une taille, et un mode d'hébergement.
- 3 Chiffrer grossièrement la latence acceptable et le volume attendu.
- 4 Identifier la donnée sensible éventuellement en jeu.
- 5 Défendre le choix devant la classe en trois minutes.

Objectif

S'entraîner à justifier un choix technique par des contraintes, pas par la mode.

Panorama et choix

- ✓ Une famille de modèles par type de tâche : génération, embedding, vision, audio.
- ✓ Poids ouverts ne veut pas dire open source : lisez la licence.
- ✓ La qualité plafonne quand la latence et le coût explosent.
- ✓ On choisit sur cinq critères, et on valide sur ses propres cas.